

10_context&memory 大纲

1、概述	1
1.1 为什么需要记忆(Memory)	1
情况1：不同messages	1
情况2：同一个messages列表	2
1.2 如何解决记忆问题	3
1.2.1 上下文工程	3
1.2.2 上下文类型及相关的API	3
1.3 LangChain的记忆	3
1.3.1 记忆的分类	4
1.3.2 记忆的管理	4
2、短期记忆	4
2.1 基于内存的持久化器	5
2.1.1 举例1：没有记忆	5
2.1.2 举例2：拥有记忆	5
步骤1：第一二轮对话	5
步骤2：第三轮对话	8
步骤3：更新线程ID	12
2.1.3 关键步骤说明	13
场景1：多用户聊天	13
场景2：同一用户的不同任务	14
2.1.4 工作原理	14
2.1.5 常见问题	15
1、为什么 Agent 不记得？	15
2、InMemorySaver 会丢失数据吗？	15
3、内存会无限增长吗？	15
4、如何清空某个会话的历史？	16
2.2 基于外部存储介质的持久化器	16
2.2.1 数据库环境准备	16
2.2.2 代码实现	16
2.2.3 查看持久化数据	20
需求1：查看所有数据库	20
需求2：查看所有schema	20
需求3：查看当前所处的schema	21
需求4：查看当前schema下的所有表	21
2.3 对比两种方式	21
举例1：基于内存存储	21
举例2：基于外部存储器存储	23
2.4 记忆治理策略（上下文管理）	27
2.4.1 消息裁剪	28
2.4.2 消息删除	31
2.4.3 摘要	35

2.4.4 自定义过滤策略	39
2.5 了解：state的理解	44
3、长期记忆	47
3.1 基本理解	47
3.1.1 什么是长期记忆	47
3.1.2 类型划分	48
3.1.3 存储架构	49
3.2 基础API的使用	50
3.2.1 put()/get()：写入/读取API	50
① put()源码剖析	50
② get()源码剖析	51
③ 举例1：基于InMemoryStore	51
举例1：添加	51
举例2：更新	52
④ 举例2：基于PostgresStore	52
举例1：添加	52
举例2：更新	52
3.2.3 search()：检索API	53
① 源码剖析	53
② 举例1：按照namespace前缀搜索	53
③ 举例2：按照filter过滤	55
④ 举例3：按照语义搜索	56
举例1：自定义嵌入函数	56
举例2：使用嵌入模型	58
3.3 在Agent运行图中访问长期记忆	60
3.3.1 在工具中访问长期记忆	60
① 基于InMemoryStore	60
② 基于PostgresStore	62
3.3.2 在中间件中访问长期记忆	65
① Node-style hooks中访问	65
② Wrap-style hooks中访问	65
1. wrap_model_call	65
2. wrap_tool_call	66
3.4 何时写入记忆	67
1. 在主流程里写（hot path）	67
2. 在后台写（background）	67
4、课后阅读：Static runtime Context	67
4.1 中间件中访问	68
4.1.1 Node-style hooks	68
4.1.2 Wrap-style hooks	71
① wrap_model_call	71
② wrap_tool_call	76
4.1.3 特殊的便捷中间件dynamic_prompt	79
4.2 工具中访问	83